Group B

Ahmed Mohamed Sayed Ahmed Othman Elghetany
Mahmoud Nasser Mahmoud Elmoghany
Haytham Mounir Mohamed
Ahmed Elsayed Moussa

DEPI BIG DATA ENGINEERING · 2026

Egypt E-Commerce Big Data Personalization Pipeline

Olist E-Commerce Batch Data Pipeline

End-to-end batch and real-time data architecture for an Egypt-localized e-commerce dataset, spanning ingestion, distributed processing, warehousing, orchestration, and dual-surface visualization.

Group ID	**ONL4-AIS6-G1 Huawei Big Data Engineer**
Stack	Kafka · Spark · PostgreSQL · Airflow · Streamlit · Power BI
Infrastructure	Docker · 13 containers,WSL2
Phases delivered	Batch ETL · Streaming ETL · Dual dashboards · Cloud integration
Storage	HDFS (Hadoop) + PostgreSQL
Architecture	CSV → Bronze (HDFS Parquet) → Silver (cleaned Parquet) → Gold (PostgreSQL
Virtualization	Microsoft Power BI Desktop, Streamlit live
Datamarts	**Delivery Performance | Customer Churn**

9	~360 K	3s / 5s
silver tables migrated	rows processed in batch	dashboard refresh / stream micro-batch

Table of Contents

1. Executive Summary

This document describes the design, implementation, and operation of an end-to-end big-data pipeline built around an Egypt-localized e-commerce dataset. The system addresses two analytical surfaces simultaneously: a **batch warehouse** built on cleaned silver-layer CSV files, and a **real-time streaming pipeline** simulating live order events. Both surfaces converge on a single PostgreSQL backbone, from which two dashboards (Streamlit and Power BI) consume identical data.

This project implements a complete batch data engineering pipeline for an Egyptian adaptation of the Olist e-commerce dataset. The original Brazilian Olist dataset was localized with Egyptian city names, governorates, and currency (EGP). The pipeline ingests raw CSV data, processes it through a medallion architecture (Bronze → Silver → Gold), loads it into a dimensional model in PostgreSQL, and visualizes it through Power BI dashboards.

Key deliverables

- Batch ETL pipeline: 9 silver-layer tables cleaned, type-cast, and persisted to PostgreSQL and Parquet.
- Streaming ETL pipeline: Python producer □ Kafka topic □ Spark Structured Streaming □
- PostgreSQL.
- Live Streamlit dashboard polling Postgres every 3 seconds with KPIs, charts, and live feed.
- Power BI streaming dataset pushed via REST API, offering an enterprise BI surface on the same data.
- Reproducible Docker-Compose infrastructure: 13 containers covering compute, storage, orchestration, and observability.
- Cloud-extension blueprint: optional Azure Event Hubs, Azure App Service, and Power BI Service.
- Complete documentation: this report, the accompanying slide deck, and code repository.
- Design and implement a production-grade batch data pipeline
- Apply the medallion lakehouse architecture (Bronze / Silver / Gold)
- Build two analytical datamarts using star schema dimensional modeling
- Orchestrate the pipeline end-to-end with Apache Airflow
- Deliver actionable business insights through Power BI dashboards

Dataset

File	Description	Rows
eg_customers_dataset.csv	Customer profiles with Egyptian governorates	99,441
eg_orders_dataset.csv	Order lifecycle with timestamps	99,441
eg_order_items_dataset.csv	Line items per order with EGP prices	112,650
eg_order_payments_dataset.csv	Payment transactions in EGP	103,886
eg_order_reviews_dataset.csv	Customer reviews and ratings	99,224
eg_products_dataset.csv	Product catalog with dimensions/weights	32,951
eg_sellers_dataset.csv	Seller profiles with Egyptian governorates	3,095
eg_geolocation_dataset.csv	Zip code to lat/lon mapping	1,000,163
eg_product_category_translation.csv	Category names in English and Arabic	71

2. Problem Statement & Objectives

2.1 Why this project

Egypt's e-commerce market is growing rapidly, but most analytics platforms target USD pricing and Western shipping zones. Local merchants need analytics that are **EGP-localized**, **governorate-aware**, and **bilingual (Arabic / English)** — and they need both historical and live signal in the same workflow. Most graduation projects stop at the batch layer; this one intentionally goes further by adding a fully working streaming layer on top of the same data.

Pain points addressed

Pain point	Why it matters	Our response
Western-centric analytics tools	Pricing and shipping default to USD, no governorate join	All monetary fields in EGP; customers joined with Egyptian governorates
Live monitoring is SaaS-dominated	Real-time dashboards usually require paid platforms	Open-source Kafka + Spark + Streamlit stack runs locally
Schema drift between Arabic and English categories	Translation tables are often missing or stale	Persistent category_translation table joins both at query time
Graduation projects rarely include streaming	Limits employment readiness	Sub-10 s end-to-end stream demonstrates production-style pattern

Concrete objectives

100 % Coverage of the original 9-table silver layer (migrated, audited, persisted).

< 8 s End-to-end latency from event production to dashboard refresh.

0 $ Baseline running cost; cloud add-ons are explicitly opt-in.

2 Independent dashboard surfaces (Streamlit + Power BI) on the same backend.

1 Reproducible Docker Compose file describing the entire local stack.

3. System Architecture Overview

The architecture has three horizontal layers. The **batch layer** handles historical silver-layer CSVs and produces both Parquet long-term storage and a clean PostgreSQL warehouse. The **streaming layer** replays real orders as live events through Kafka, processes them with Spark Structured Streaming, and writes rolling aggregates into dedicated PostgreSQL tables. The **presentation layer** hosts both Streamlit (engineer-friendly) and Power BI (business-friendly) consumers, plus pgAdmin for ad-hoc SQL.

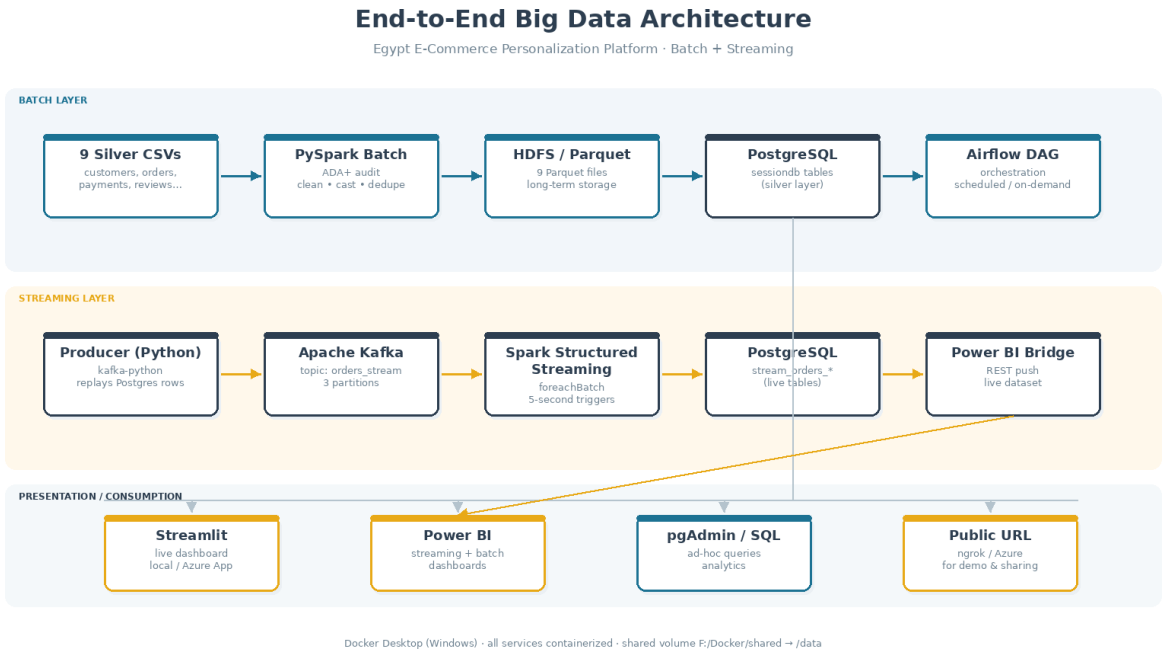

Figure 1 — End-to-end architecture. Both batch and streaming paths converge on PostgreSQL, which is the single source of truth for every dashboard.

Architectural decisions

PostgreSQL as the convergence point. Both batch and streaming layers terminate in the same database. This single source of truth eliminates dual-write inconsistency and means dashboards never need to know which pipeline produced any given row.

Kafka in KRaft mode (no Zookeeper). The Confluent cp-kafka image supports KRaft, removing a moving part from the cluster. Suitable for single-broker development; production would add 3 brokers and use SASL.

Spark Structured Streaming over Spark Streaming (DStreams). Structured Streaming provides exactly-once semantics with checkpointing and is the modern API. DStreams is deprecated.

Shared Docker volume for code + data. Mounting F:/Docker/shared as /data inside every container makes producer and consumer scripts, Parquet files, and SQL files instantly available to all services without rebuilding images.

4.Technology Stack & Infrastructure

All thirteen services run as Docker containers, orchestrated by Docker Compose on a Windows host. Heavy components (Spark, Hadoop, Kafka) use public images, while application code is mounted from the shared volume so it can be edited in place without rebuilding any image.

The entire infrastructure runs locally on Windows using Docker Compose (WSL2 backend). All services communicate over a shared Docker network (airflow-network).

Shared Volume Structure

```
D:\Olist_Project\
  ├── dags\                    ← Airflow DAG files
  ├── shared\                   ← Mounted as /data inside containers
  │   ├── data\                 ← 9 source CSV files
  │   ├── scripts\
  │   │   ├── bronze\bronze_ingestion.py
  │   │   ├── silver\silver_transformation.py
  │   │   └── gold\
  │   │       ├── gold_delivery_performance.py
  │   │       └── gold_customer_churn.py
  │   └── jars\postgresql-42.7.3.jar
  ├── logs\                    ← Airflow logs
  └── docker-compose.yaml
```

HDFS Directory Structure

```
/olist/
  ├── bronze/   {customers, geolocation, order_items, order_payments,
  │              order_reviews, orders, product_category_translation,
  │              products, sellers}
  ├── silver/   {same 9 folders — cleaned Parquet}
  └── gold/
      ├── dim_date, dim_customer, dim_seller, dim_product
      ├── fct_order_delivery, fct_seller_fulfillment
      └── customer_churn/
          ├── dim_date, dim_product, dim_customer_profile
          └── fct_customer_orders, fct_churn_summary
```

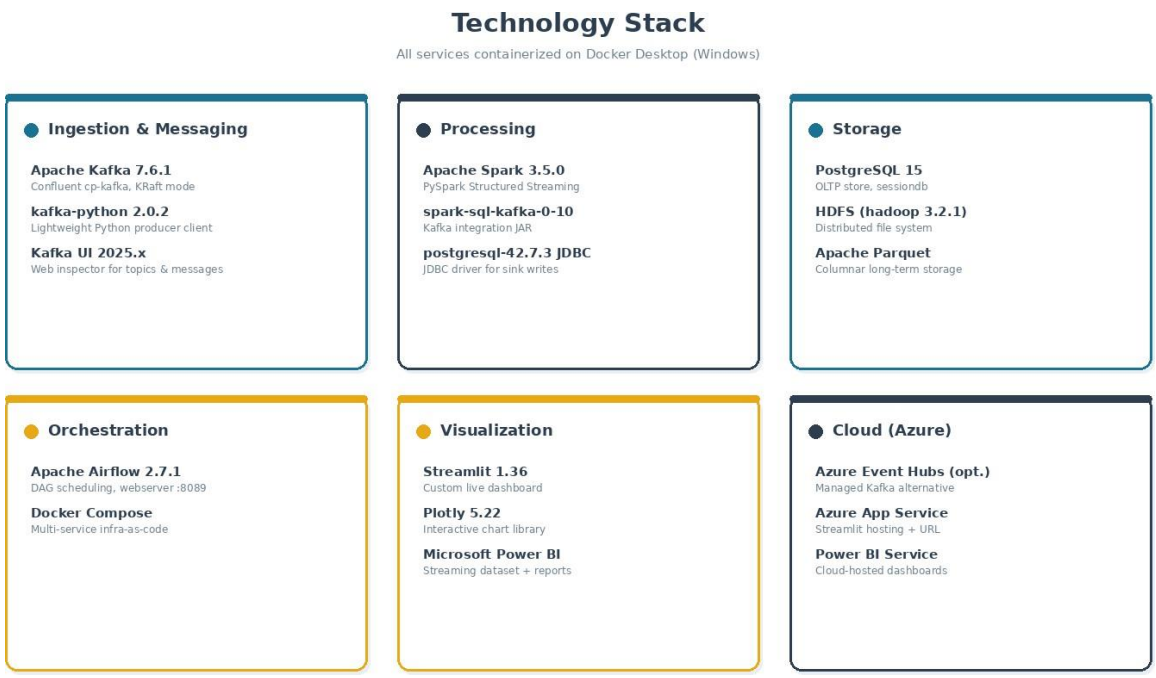

Figure 2 — Technology stack summary by category.

Container inventory

Container	Image	Port(s)	Role
jupyter	jupyter/pyspark-notebook:spark-3.5.0	8888	Notebook + spark-submit host
broker	confluentinc/cp-kafka:7.6.1	9092 / 29092 / 9101 / 1234	Kafka (KRaft) + JMX exporter
kafka-ui	provectuslabs/kafka-ui:latest	8090	Topic browser
namenode	bde2020/hadoop-namenode:2.0.0-hadoop3.2.1-java8	9000	HDFS NameNode
datanode	bde2020/hadoop-datanode:2.0.0-hadoop3.2.1-java8	9864	HDFS DataNode
postgres_general	postgres:15	5432	OLTP / warehouse
kafka-ui	Kafka UI	localhost:8080	kafka-ui
spark	Spark Master (3.5.0)	localhost:8080 (Spark UI)	spark
spark-worker	Spark Worker (1 node)	Registers with master	spark-worker
jupyter	Jupyter Notebook	localhost:8888	jupyter
namenode	HDFS NameNode	localhost:9870 (WebHDFS)	namenode
datanode	HDFS DataNode	Registers with namenode	datanode

Shared Volume Structure

```
D:\Olist_Project\
  ├── dags\                    ← Airflow DAG files
  ├── shared\                   ← Mounted as /data inside containers
  │   ├── data\                 ← 9 source CSV files
  │   ├── scripts\
  │   │   ├── bronze\bronze_ingestion.py
  │   │   ├── silver\silver_transformation.py
  │   │   └── gold\
  │   │       ├── gold_delivery_performance.py
  │   │       └── gold_customer_churn.py
  │   └── jars\postgresql-42.7.3.jar
  ├── logs\                     ← Airflow logs
  └── docker-compose.yaml
```

HDFS Directory Structure

```
/olist/
  ├── bronze/   {customers, geolocation, order_items, order_payments,
  │              order_reviews, orders, product_category_translation,
  │              products, sellers}
  ├── silver/   {same 9 folders — cleaned Parquet}
  └── gold/
     ├── dim_date, dim_customer, dim_seller, dim_product
     ├── fct_order_delivery, fct_seller_fulfillment
     └── customer_churn/
          ├── dim_date, dim_product, dim_customer_profile
          └── fct_customer_orders, fct_churn_summary
```

Bronze Layer — Raw Ingestion

Script: /data/scripts/bronze/bronze_ingestion.py

The Bronze layer reads each CSV from the shared volume using the file:// prefix (required because defaultFS is set to HDFS) and writes each as Parquet to HDFS with mode='overwrite'. No transformations are applied — this is a landing zone.

Key Implementation Notes

file:// prefix required when defaultFS=hdfs://namenode:9000 to force reading from local disk

inferSchema=true and encoding=UTF-8 preserve Arabic characters in category and governorate columns

All 9 files written as snappy-compressed Parquet partitions

Row Counts (Post-Ingestion)

Table	Rows Ingested
customers	99,441
geolocation	1,000,163
order_items	112,650
order_payments	103,886
order_reviews	99,224
orders	99,441
product_category_translation	71
products	32,951
sellers	3,095

Run Command

```
docker exec -it spark spark-submit --master spark://spark:7077 --conf spark.hadoop.fs.defaultFS=hdfs://namenode:9000 /data/scripts/bronze/bronze_ingestion.py
```

Silver Layer — Cleaning & Transformation

Script: /data/scripts/silver/silver_transformation.py

The Silver layer reads Bronze Parquet files, applies EDA-driven cleaning and standardization, and writes cleaned Parquet back to HDFS. All operations use pure Spark SQL / DataFrame API (no Python UDFs) to avoid Python version mismatch issues.

Transformations Applied

Customers

- Standardized customer_city (lowercase trim), customer_state (uppercase), customer_governorate (initcap)
- Dropped fully duplicate rows and rows with null customer_id or customer_unique_id

Geolocation

- Standardized city and state fields
- Removed rows with null latitude or longitude
- Deduplicated by (zip_code_prefix, lat, lng) — removed ~2 redundant rows

Orders

- Cast all 5 timestamp columns to TimestampType
- Removed rows with null order_purchase_timestamp
- Nulled out 1,359 chronological violations: carrier_date < approved_at → null carrier_date; customer_delivery < carrier_date → null customer_delivery
- Standardized order_status to lowercase

Products

- Dropped 610 rows where product_category_name was null (entire row was null)
- Fixed column name typos: product_name_lenght → product_name_length, product_description_lenght → product_description_length
- Filled 2 null numeric rows with column medians
- Computed product_volume_cm3 = length × height × width

Product Category Translation

- Added 2 missing categories using Spark SQL UNION ALL (not createDataFrame): pc_gamer and portateis_cozinha_e_preparadores_de_alimentos
- Standardized all category names to lowercase

Order Items

- Filtered rows where price_egp <= 0 or freight_value_egp < 0
- Dropped duplicates and null order/product/seller IDs

Order Payments

- Filtered zero-value payments (9 rows removed)
- Standardized payment_type to lowercase

Order Reviews

- Filled 87,656 null review_comment_title values with 'No Title'
- Filled 58,247 null review_comment_message values with 'No Comment'
- Filtered review_score not between 1 and 5

Silver Row Counts

Table	Bronze Rows	Silver Rows	Delta
customers	99,441	99,441	0
geolocation	1,000,163	1,000,161	-2 (duplicate lat/lng)
orders	99,441	99,441	0 (violations nulled, not dropped)
order_items	112,650	112,650	0
order_payments	103,886	103,877	-9 (zero-value)
order_reviews	99,224	99,224	0
products	32,951	32,341	-610 (null category rows)
sellers	3,095	3,095	0
product_category_translation	71	73	+2 (missing categories added)

Gold Layer — Delivery Performance Datamart

Script: /data/scripts/gold/gold_delivery_performance.py

Schema: delivery_performance (PostgreSQL: sessiondb)

The Delivery Performance datamart is a star schema with 2 fact tables and 4 dimension tables. It enables analysis of order delivery speed, on-time rates, seller fulfillment efficiency, and freight cost patterns.

Dimensional Model

dim_date (800 rows)

Date spine covering 2016-09-04 to 2018-11-12. Columns: date_sk (PK), full_date, day_number, day_name, week_number, month_number, month_name, quarter_number, year_number, is_weekend, is_month_start, is_month_end.

dim_customer (99,441 rows)

One row per customer_id (not unique customer). Columns: customer_sk (PK), customer_id, customer_unique_id, customer_zip_code_prefix, customer_city, customer_state, customer_region (governorate), latitude, longitude. Lat/lon joined from geolocation average per zip.

dim_seller (3,095 rows)

One row per seller. Columns: seller_sk (PK), seller_id, seller_zip_code_prefix, seller_city, seller_state, seller_region, latitude, longitude.

dim_product (32,341 rows)

One row per product. Columns: product_sk (PK), product_id, product_category_name, product_category_english, product_weight_g, product_length_cm, product_height_cm, product_width_cm, product_volume_cm3, product_photos_qty, product_name_length, product_description_length, logistics_size_category (small/medium/large/extra_large), logistics_weight_category (light/medium/heavy/very_heavy).

fct_order_delivery (98,666 rows)

One row per order. Key metrics: delivery_duration_days, delay_days, buffer_days, delivery_status (on_time/late/very_late/not_delivered), on_time_flag, distance_bucket (0-50km/50-200km/200-500km/500km+), freight_total_value, total_items_count, seller_count, is_multi_seller_order.

fct_seller_fulfillment (112,650 rows)

One row per order line item. Key metrics: freight_value, item_price, product_weight_g, product_volume_cm3, seller_preparation_days, shipping_deadline_gap_days, freight_ratio (freight/price), heavy_product_flag, oversized_product_flag, seller_to_customer_distance_km (Haversine formula).

Key Computed Fields

Field	Table	Logic
delivery_status	fct_order_delivery	delay_days ≤ 0 → on_time; 1-7 → late; null → not_delivered; >7 → very_late
distance_bucket	fct_order_delivery	Haversine distance bucketed: 0-50/50-200/200-500/500+ km
delay_days	fct_order_delivery	datediff(actual_delivery, estimated_delivery)
freight_ratio	fct_seller_fulfillment	freight_value / item_price
logistics_size_category	dim_product	volume ≤1000cm³=small; ≤10000=medium; ≤50000=large; else extra_large
logistics_weight_category	dim_product	weight ≤500g=light; ≤2000=medium;

		≤10000=heavy; else very_heavy

Auto-Truncate Mechanism

The Gold script uses psycopg2 (delivery) and py4j DriverManager (churn) to truncate all tables at startup with RESTART IDENTITY CASCADE before writing, ensuring idempotent re-runs.

HDFS Permissions Fix

The script uses WebHDFS REST API calls via subprocess.run (curl to namenode:9870) to set chmod 777 on gold directories before writing. This prevents permission failures when the script is submitted by different users (spark vs airflow).

Gold Layer — Customer Churn Datamart

Script: /data/scripts/gold/gold_customer_churn.py

Schema: customer_churn (PostgreSQL: sessiondb)

The Customer Churn datamart uses a behavioral churn definition: a customer who made their last purchase more than 180 days before the dataset end date (2018-10-17) is considered churned.

Churn Segmentation Logic

Segment	Definition	Count
one_time	total_orders == 1 (regardless of recency)	~93,111
active	total_orders > 1 AND days_since_last_order ≤ 180	~975 (loyal)
churned	total_orders > 1 AND days_since_last_order > 180	~2,010
loyal	Subset of active: multi-purchase + recent	~975

Dataset end date: 2018-10-17 (max order_purchase_timestamp). Churn threshold: 180 days.

Dimensional Model

dim_date (774 rows)

Date spine from first to last order purchase date. Same structure as delivery dim_date.

dim_product (32,341 rows)

Identical structure to delivery dim_product — a self-contained copy in the customer_churn schema.

dim_customer_profile (96,096 rows)

One row per customer_unique_id (not customer_id). Columns include: first_order_date, last_order_date, total_orders, total_spend_egp, avg_order_value_egp, days_since_first_order, days_since_last_order, customer_segment, churn_flag. Lat/lon joined from geolocation.

fct_customer_orders (99,441 rows)

One row per order. Includes: order_sequence_number (1st, 2nd... purchase per customer), days_since_previous_order, payment_value_egp, items_count, distinct_categories, delivery_status, review_score.

fct_churn_summary (96,096 rows)

One row per unique customer — summary fact table. Includes: customer_segment, churn_flag, total_orders, total_spend_egp, avg_order_value_egp, avg_review_score, avg_days_between_orders, days_since_last_order, top_category (most purchased English category name).

Key Business Metrics

Total unique customers: 96,096

One-time customers: 96.88% — almost all customers only ever bought once

Loyal customers (multi-purchase + recent): 975 (1.01%)

Churned customers: 2,010 (2.09%)

Average repeat purchase rate: 3.12%

Loyal customers avg spend: ~3,200 EGP vs one-time: ~1,500 EGP

Batch ETL Phase

The batch phase reads 9 silver-layer CSVs covering customers, orders, order items, payments, reviews, products, sellers, geolocation, and category translation. Each is audited for nulls and duplicates, type-cast where needed, and persisted in two formats simultaneously: **Parquet** for long-term storage in the data lake and **PostgreSQL** for fast SQL access.

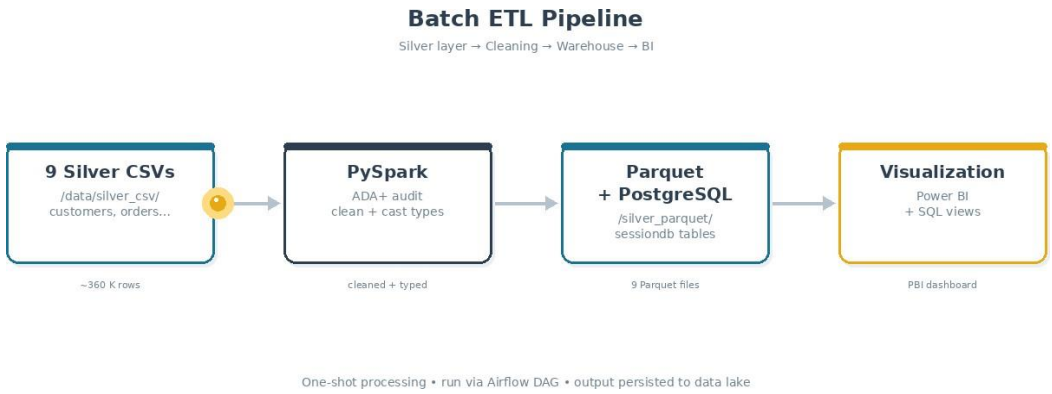

Figure 3 — Batch ETL flow: source files → cleaning → persistent storage → BI.

2.2 Stages performed

Ingestion. Read 9 CSVs from /data/silver_csv/ into Spark DataFrames using inferSchema with explicit overrides where needed.

ADA+ Audit. Custom audit step counted nulls per column and detected duplicate keys across customer, order, and review tables.

Type casting. Five date columns in orders converted STRING → TIMESTAMP. review_score converted STRING → INTEGER. payment_installments cast to INT.

Data cleaning. 3,373 invalid review rows removed (null review_id or order_id). Money fields renamed to ..._egp to make EGP localization explicit.

Persistence. All 9 tables saved as Parquet to /data/silver_parquet/ and loaded into PostgreSQL sessiondb via SQLAlchemy + psycopg2.

Export. Clean CSV copies dumped to /data/export_csv/ for offline analysis.

2.3 Row counts after cleaning

Table	Rows	Notes
customers	99,441	Joined with customer_governorate / governorate_ar
orders	99,441	5 timestamp columns cast; is_delivered flag derived
order_items	112,650	EGP price + freight value
payments	103,886	Installments cast to int
reviews	100,789	3,373 invalid rows removed

products	32,951	Category + dimensions
sellers	3,095	Governorate joined
geolocation	9,637	Egypt-localized coordinates
category_translation	74	Arabic + English category mapping

2.4 Persistent schema (excerpt)

Below is the key column subset that drives both batch and streaming joins.

```
customers      : customer_id, customer_unique_id, customer_zip_code_prefix,
                 customer_city, customer_state, customer_governorate,
                 customer_governorate_ar
orders         : order_id, customer_id, order_status,
                 order_purchase_timestamp        (timestamp),
                 order_delivered_customer_date   (timestamp),
                 order_estimated_delivery_date   (timestamp),
                 is_delivered
order_items    : order_id, order_item_id, product_id, seller_id,
                 price_egp (double), freight_value_egp (double)
payments       : order_id, payment_sequential, payment_type,
                 payment_installments (int), payment_value_egp (double)
```

2.5 Quality assurance checks

- **Null detection.** Counted nulls in every column; rows missing critical IDs were removed.
- **Duplicate scan.** Dropped exact duplicate order_id and review_id rows.
- **Range validation.** Verified review_score is in [1, 5] and payment_value_egp is non-negative.
- **Referential integrity.** Verified every order_id in order_items, payments, and reviews exists in orders.
- **Timezone normalization.** All timestamps stored as UTC; presentation layer converts to Africa/Cairo.

2.6 Persistence formats compared

Format	Used for	Pros	Cons
Parquet	Long-term cold storage, Spark re-reads	Columnar, compressed (~5× smaller than CSV); schema embedded; predicate pushdown	Not human-readable; needs Spark/pandas to inspect
PostgreSQL	OLTP queries, dashboards, Streamlit	Strong typing; indexed lookups; familiar SQL surface for downstream consumers	Single-node by default; less efficient for full scans on huge tables
CSV export	Sharing with non-technical reviewers	Universally readable in Excel/Sheets	No types; large files; quoting edge cases

7. Airflow Orchestration

DAG file: D:\Olist_Project\dags\olist_pipeline_dag.py

The Airflow DAG orchestrates the full pipeline. Airflow submits spark-submit commands directly (not via docker exec) since the scheduler container has PySpark installed and shares the airflow-network with the Spark cluster.

7.1 DAG Structure

DAG ID: olist_batch_pipeline
Schedule: @daily
Retries: 1 (5-minute delay)

Task Order (Sequential):
bronze_ingestion → silver_transformation → gold_delivery_performance → gold_customer_churn

Note: Gold tasks run sequentially (NOT parallel) to prevent the single Spark worker from being overloaded by two concurrent heavy jobs.

7.2 Task Configuration

Task ID	Timeout	Script
bronze_ingestion	1 hour	/data/scripts/bronze/bronze_ingestion.py
silver_transformation	1 hour	/data/scripts/silver/silver_transformation.py
gold_delivery_performance	2 hours	/data/scripts/gold/gold_delivery_performance.py
gold_customer_churn	2 hours	/data/scripts/gold/gold_customer_churn.py

1. 7.3 Key Lessons Learned (Infrastructure)

2. JAVA_HOME must be exported explicitly in bash_command for spark-submit to work from Airflow: export JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64
3. JAR path: Always /data/jars/postgresql-42.7.3.jar (shared volume) — never /opt/spark/jars/ which is only accessible from the Spark container
4. Python version mismatch: Airflow=Python 3.9, Spark worker=Python 3.11. Never use spark.createDataFrame() from Python objects — always use pure Spark SQL UNION ALL
5. file:// prefix required for local CSV reads when defaultFS=hdfs
6. HDFS permissions: Both Gold scripts auto-run WebHDFS chmod 777 at startup via curl
7. Gold tasks in parallel overload the single Spark worker — must run sequentially

8. Power BI Dashboards

Data source: 11 CSV files exported from PostgreSQL to D:\Olist_Project\powerbi_data\

File: Olist_Delivery_Performance.pbix

All column types fixed in Power Query Editor before building measures. Boolean columns converted to True/False, numeric to Whole Number or Decimal.

8.1 Data Model Relationships

Delivery Performance Datamart

From (Dim)	Column	To (Fact)	Column	Status
dim_customer	customer_sk	fct_order_delivery	customer_sk_fk	Active
dim_seller	seller_sk	fct_order_delivery	seller_sk_fk	Active
dim_date	date_sk	fct_order_delivery	purchase_date_sk	Active
dim_date	date_sk	fct_order_delivery	estimated_delivery_date_sk	Inactive
dim_date	date_sk	fct_order_delivery	actual_delivery_date_sk	Inactive
dim_seller	seller_sk	fct_seller_fulfillment	seller_sk_fk	Active
dim_product	product_sk	fct_seller_fulfillment	product_sk_fk	Active
dim_customer	customer_sk	fct_seller_fulfillment	customer_sk_fk	Inactive
dim_date	date_sk	fct_seller_fulfillment	purchase_date_sk	Active
dim_date	date_sk	fct_seller_fulfillment	shipping_limit_date_sk	Inactive

8.2 DAX Measures

```
On-Time Rate = DIVIDE(CALCULATE(COUNTROWS(fct_order_delivery),
fct_order_delivery[on_time_flag] = TRUE()), CALCULATE(COUNTROWS(fct_order_delivery),
fct_order_delivery[on_time_flag] <> BLANK()), 0)

Avg Delivery Days = AVERAGE(fct_order_delivery[delivery_duration_days])

Avg Delay Days = CALCULATE(AVERAGE(fct_order_delivery[delay_days]),
fct_order_delivery[delay_days] > 0)

Total Orders = COUNTROWS(fct_order_delivery)
Late Orders = CALCULATE(COUNTROWS(fct_order_delivery),
fct_order_delivery[delivery_status] = "late" || fct_order_delivery[delivery_status] = "very_late")
```

8.3 Dashboard Pages — Delivery Performance

1. **Page 1: Delivery Performance Overview**
2. KPI Cards: 99K Total Orders | 12 Avg Delivery Days | 10 Avg Delay Days | 89.9% On-Time Rate
3. Donut Chart: Orders by delivery status — on_time (89.9%), late (4.5%), very_late, not_delivered
4. Bar Chart: On-Time Rate by Customer Region — Luxor 95.5%, North Sinai 95.1%, Cairo 92.2%
5. Line/Column Chart: Monthly Orders vs On-Time Rate — peak August 10.7K orders, lowest Feb 80.6%

on-time

6. Bar Chart: Avg Delivery Days by Distance Bucket — 0-50km: 11 days, 500km+: 18 days
7. Bar Chart: Top 10 Product Categories by Total Freight Value — bed_bath_table 1.9M EGP leads

8. **Page 2: Delivery Deep Dive**

9. Map: Customer Locations by Delivery Status (bubble size = order count)
10. Bar Chart: Avg Delay Days by Seller Region — Suez highest (~20 days), Sharqia lowest
11. Bar Chart: Avg Delay Days by Customer Region — New Valley highest (~50 days, most remote)
12. Bar Chart: On-Time Rate by Year and Quarter — 2016: ~75%, 2017: ~90%, 2018: ~95% (improving)
13. Column Chart: Delivery Duration and On-Time Rate by Distance

14. **Page 3: Seller Fulfillment Analysis**

15. KPI Cards: 110.24 Avg Distance KM | 21.39M Total Freight | 32.1% Avg Freight Ratio | 5K Heavy Orders
16. Bar Chart: Top 10 Sellers by Freight Value — top seller: 0.49M EGP
17. Donut Chart: Orders by Product Weight Category — light 41.34%, medium 34.98%, heavy 17.62%
18. Bar Chart: Avg Freight Ratio by Category — home_comfort_2 anomaly at 93.4% (freight = product price)
19. Bar Chart: Avg Preparation Days by Seller Region — Monufia 8.5 days slowest, Luxor 6.7 days fastest
20. Scatter Plot: Distance vs Freight Value by Product Size

21. 8.4 Dashboard Pages — Customer Churn

22. **Page 1: Churn Overview**

23. KPI Cards: 96K Customers | 3.1% Repeat Purchase Rate | 1.58K Avg Spend | 2.1% Churn Rate
24. Stacked Bar: Customer Segment Distribution by Region
25. Donut Chart: Customer Segments — 96.88% one_time, 2.09% churned, 1.01% loyal
26. Bar Chart: Customers by Region
27. Line Chart: New Customers Acquired by Month

28. **Page 2: Customer Behavior**

29. KPI Cards: 80.08 Avg Days Between Orders | 2K Churned | 975 Loyal | 93K One-Time
30. Bar Chart: Top 10 Product Categories for Churned Customers
31. Bar Chart: Avg Spend by Segment — loyal 3.2K EGP, one_time 1.5K EGP
32. Bar Chart: Avg Review Score by Segment
33. Bar Chart: Avg Days Between Orders by Segment — churned 57 days, loyal 129 days

34. **Page 3: Geographic Churn**

35. KPI Cards: 96K Customers | 1.58K Avg Spend | 2.1% Churn Rate
36. Bar Chart: Churn Rate by Region — North Sinai highest 5.2%, most regions ~2%
37. Stacked Bar: Customer Segment Mix for Top 5 Regions
38. Bar Chart: Total Spend by Region — Cairo 57M EGP dominates

40. 9. Key Business Insights

41. 9.1 Delivery Performance Insights

42. Overall on-time delivery rate: 89.9% across 99K orders
43. Significant improvement over time: 2016 (~75%) → 2017 (~90%) → 2018 (~95%) — the business was rapidly maturing
44. February consistently worst month for delivery (80.6% on-time) — possible seasonal or operational factor
45. Remote governorates severely impacted: New Valley customers experience ~50-day delays when late
46. Suez-based sellers cause the most delays (~20 avg delay days) despite being geographically central
47. Distance correlates strongly with delivery time: 0-50km = 11 days avg vs 500km+ = 18 days
48. Home_comfort_2 category: 93.4% freight ratio — shipping costs nearly equal the item price (anomaly, possibly data quality)
49. Monufia sellers have the slowest preparation time (8.5 days) — opportunity for seller performance management

50. 9.2 Customer Churn Insights

51. 96.88% of customers only ever purchased once — extreme one-time purchase problem
52. Only 975 loyal customers (repeat buyers, still active) out of 96,096 unique customers
53. Loyal customers spend 2.1x more on average than one-time buyers (3,200 vs 1,500 EGP)
54. Churned customers had 57 days avg between their (few) orders; loyal customers had 129 days — suggesting churned customers had erratic behavior
55. North Sinai has highest churn rate at 5.2% — possibly related to delivery issues (high delay rates)
56. Cairo generates 57M EGP of total revenue — retention programs here have highest potential ROI

58. 10. Technical Decisions & Lessons Learned

59. 10.1 Architecture Decisions

60. Chose BashOperator over SparkSubmitOperator in Airflow to have full control over environment variables, specifically JAVA_HOME which SparkSubmitOperator did not handle correctly
61. Used pure Spark SQL UNION ALL instead of spark.createDataFrame() everywhere, to avoid Python version mismatch between Airflow (3.9) and Spark workers (3.11)
62. Chose CSV export for Power BI instead of direct PostgreSQL JDBC connection to avoid driver installation complexity on Windows
63. Gold tasks made sequential in the DAG (not parallel) because a single Spark worker cannot handle two concurrent heavy jobs — causes TaskResultLost block manager failures
64. Each Gold script truncates its own tables at startup (auto-truncate via JDBC) making re-runs idempotent without manual pgAdmin intervention
65. Separate PostgreSQL schemas (delivery_performance, customer_churn) with independent dimension copies for clean separation

66. 10.2 Data Quality Decisions

67. Chronological violations in orders (1,359 rows): chosen to null out the violating timestamp rather than drop the entire order, preserving the most data
68. 610 products with fully null attributes: dropped entirely as they provide no analytical value
69. Two missing product category translations: added via Spark SQL to maintain referential integrity
70. Arabic columns preserved as-is (UTF-8) in all layers — Spark, HDFS, and PostgreSQL all handle UTF-8 natively
71. home_comfort_2 freight anomaly (93.4% ratio) left in data as legitimate business finding worth noting

72. 10.3 Infrastructure Lessons

73. HDFS permissions reset on container restart: solved by building chmod 777 call into Gold scripts using WebHDFS REST API
74. Spark JAR path: /data/jars/ (shared volume) is the only path accessible from both Airflow and Spark containers
75. After computer restart: spark-worker needs stop/start cycle to reconnect cleanly to master
76. Never use docker-compose down -v — this destroys all volumes including HDFS and PostgreSQL data

78. 11. How to Run the Pipeline

79. 11.1 Start Infrastructure

```
80. cd D:\Olist_Project
81. docker-compose up -d
82. # Wait 2 minutes for all containers to initialize
```

83. 11.2 After Restart (if needed)

```
84. # Reconnect Spark worker
85. docker stop spark spark-worker
86. docker start spark
87. docker start spark-worker
88.
89. # Verify HDFS data survived
90. docker exec namenode hdfs dfs -ls /olist/silver
```

91. 11.3 Trigger Full Pipeline via Airflow

92. Open http://localhost:8089 (admin/admin)
93. Find DAG: olist_batch_pipeline
94. Click the ▶ Trigger button
95. Total runtime: approximately 20-25 minutes
96. Task order: bronze_ingestion → silver_transformation → gold_delivery_performance → gold_customer_churn

11.4 Run Individual Scripts (Manual)

```
# Bronze
docker exec -it spark spark-submit --master spark://spark:7077 --conf
spark.hadoop.fs.defaultFS=hdfs://namenode:9000 /data/scripts/bronze/bronze_ingestion.py

# Silver
docker exec -it spark spark-submit --master spark://spark:7077 --conf
spark.hadoop.fs.defaultFS=hdfs://namenode:9000 /data/scripts/silver/silver_transformation.py

# Gold Delivery
docker exec -it spark spark-submit --master spark://spark:7077 --conf
spark.hadoop.fs.defaultFS=hdfs://namenode:9000 --jars /opt/spark/jars/postgresql-42.7.3.jar
/data/scripts/gold/gold_delivery_performance.py

# Gold Churn
docker exec -it spark spark-submit --master spark://spark:7077 --conf
spark.hadoop.fs.defaultFS=hdfs://namenode:9000 --jars /opt/spark/jars/postgresql-42.7.3.jar
/data/scripts/gold/gold_customer_churn.py
```

11.5 Verify Data in PostgreSQL

```
-- Delivery Performance
SELECT 'dim_date' AS tbl, COUNT(*) FROM delivery_performance.dim_date
UNION ALL SELECT 'dim_customer', COUNT(*) FROM delivery_performance.dim_customer
UNION ALL SELECT 'dim_seller', COUNT(*) FROM delivery_performance.dim_seller
UNION ALL SELECT 'dim_product', COUNT(*) FROM delivery_performance.dim_product
```

```
UNION ALL SELECT 'fct_order_delivery', COUNT(*) FROM
delivery_performance.fct_order_delivery
UNION ALL SELECT 'fct_seller_fulfillment', COUNT(*) FROM
delivery_performance.fct_seller_fulfillment;

-- Expected: 800 | 99441 | 3095 | 32341 | 98666 | 112650
```

3. Streaming ETL Phase

The streaming phase simulates a live order feed by replaying real Postgres rows through Kafka, then uses Spark Structured Streaming to aggregate events in 5-second micro-batches and write three rolling tables back to Postgres. Streamlit polls these tables every 3 seconds to render a continuously-updated dashboard.

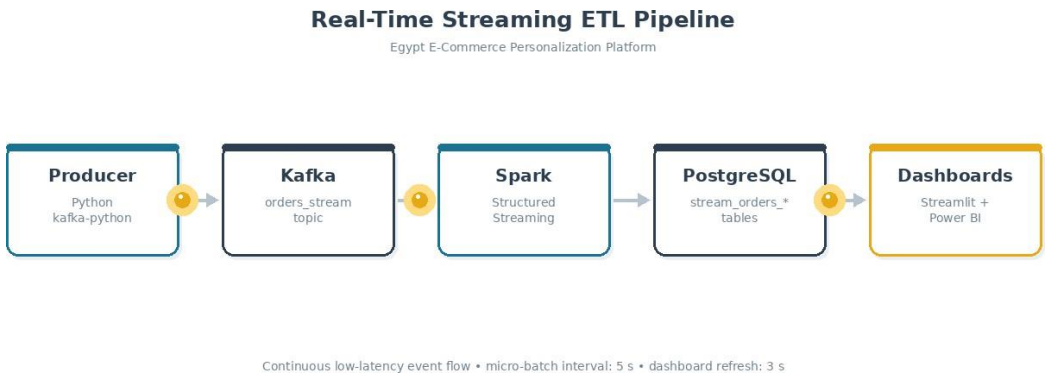

Figure 4 — Streaming pipeline flow: Producer → Kafka → Spark → Postgres → Dashboards.

3.1 Event schema

Each event sent to the **orders_stream** topic is a JSON document with the following structure:

```
{
  "order_id": "abc-123",
  "customer_id": "...",
  "customer_governorate": "Cairo",
  "order_status": "delivered",
  "product_id": "...",
  "price_egp": 245.50,
  "freight_value_egp": 18.00,
  "payment_type": "credit_card",
  "payment_installments": 3,
  "payment_value_egp": 263.50,
  "event_time": "2026-05-23T10:15:32.000+00:00"
}
```

3.2 Producer design

The producer (*producer.py*) pulls a pool of 5,000 real joined order rows from PostgreSQL into memory, then emits them to Kafka with a random delay between 0.2 and 1.5 seconds. Key design choices:

- **acks="all" + retries=5.** No message loss on transient broker hiccups.
- **Key = order_id.** Ensures all events for the same order land on the same partition (preserves per-order ordering).
- **Pool-then-replay.** Reads from Postgres once at startup; can run indefinitely without exhausting

the source.

- **Graceful SIGINT shutdown.** Flushes pending messages before exit, preventing data loss.

• **USE_LOCAL environment switch.** Same code runs from Windows host (localhost:9092) or inside Docker (broker:29092).

3.3 Consumer (Spark) design

The consumer (*consumer.py*) uses Spark Structured Streaming with a **foreachBatch** sink. This is the recommended pattern when writing to a JDBC target because Spark does not provide a native streaming JDBC sink. For each 5-second micro-batch the consumer writes:

• **stream_orders_raw.** Every raw event — used as an audit trail and as the source for the live feed table in the dashboard.

• **stream_orders_live_gov.** Per-governorate aggregates of order count and revenue, tagged with batch_id and batch_time.

• **stream_orders_live_status.** Per-status aggregates (delivered, shipped, processing, canceled, …).

3.4 Operational guarantees

Checkpointing. Spark commits Kafka offsets after each successful micro-batch to /data/checkpoints/orders_stream. On restart, processing resumes from the last committed offset, providing exactly-once semantics for the Kafka side and at-least-once for the JDBC sink (idempotent via batch_id).

Backpressure. Spark's Structured Streaming runtime automatically throttles input rate if the sink is slow, preventing memory blowups.

Restart safety. Both producer and consumer can be killed and restarted independently; Kafka's log retention means no events are lost as long as the topic retention window is not exceeded.

Idempotent appends. Each row carries batch_id and batch_time; downstream queries can deduplicate or drop replays.

3.5 Core sink code (excerpt)

The following excerpt from *consumer.py* shows how three sinks are written atomically per micro-batch.

```
def foreach_batch(batch_df, batch_id):
    if batch_df.rdd.isEmpty():
        return
    batch_df.persist()

    # 1) Raw event feed
    raw = batch_df.withColumn("batch_id", lit(batch_id))
    write_jdbc(raw, "stream_orders_raw")

    # 2) Per-governorate rollup
    gov = (batch_df
           .groupBy("customer_governorate")
           .agg(_count("order_id").alias("orders_count"),
                _sum("payment_value_egp").alias("total_revenue_egp"))
           .withColumn("batch_id",    lit(batch_id))
           .withColumn("batch_time", current_timestamp()))
    write_jdbc(gov, "stream_orders_live_gov")

    # 3) Per-status rollup
    status = (batch_df
              .groupBy("order_status")
              .agg(_count("order_id").alias("orders_count"),
                   _sum("payment_value_egp").alias("total_revenue_egp"))
              .withColumn("batch_id",    lit(batch_id))
              .withColumn("batch_time", current_timestamp()))
    write_jdbc(status, "stream_orders_live_status")

    batch_df.unpersist()
```

4. Visualization Layer

The same Postgres backbone feeds two independent dashboards: a **custom Streamlit application** for engineers and a **Power BI streaming dashboard** for business users. This dual-surface pattern is what production teams typically deploy.

4.1 Streamlit dashboard

The Streamlit app (*app.py*) polls three Postgres tables every 3 seconds and renders four KPI cards, a governorate bar chart, an order-status donut, a revenue time-series, and a live feed table.

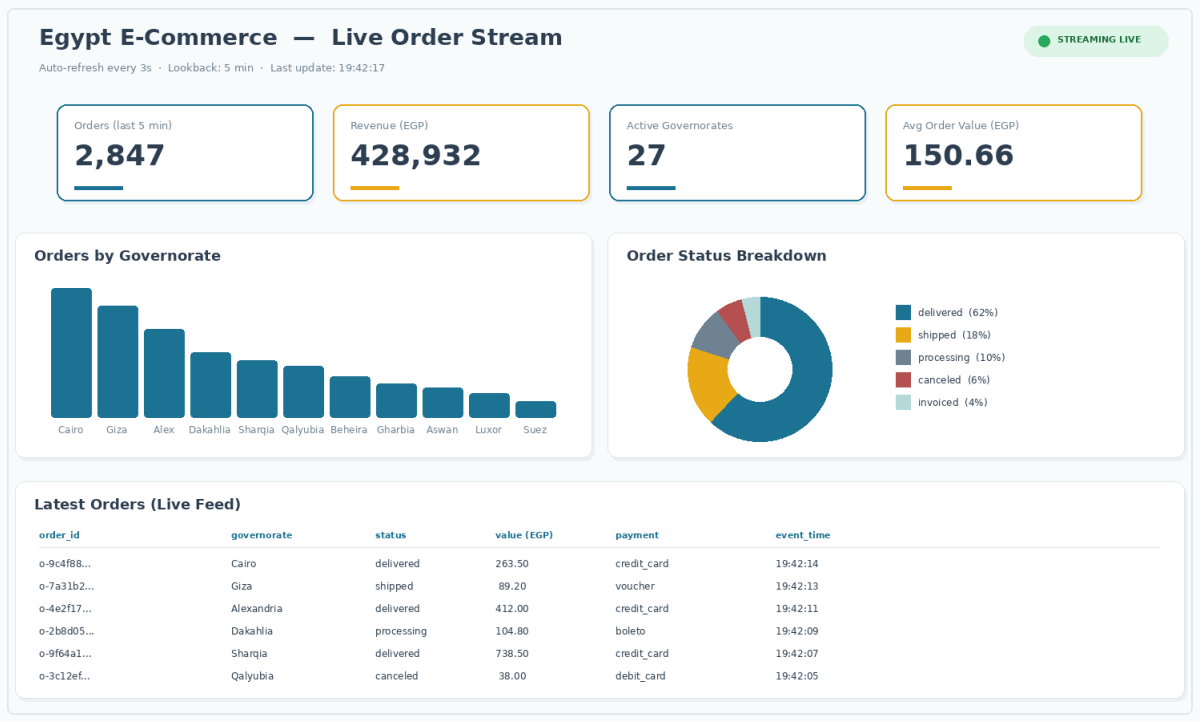

Figure 5 — Streamlit dashboard layout (mock render).

4.2 Dashboard components

Component	Source table(s)	Refresh
KPI cards	stream_orders_raw (last 5 min lookback)	3 s
Orders by Governorate	stream_orders_live_gov	3 s
Order Status (donut)	stream_orders_live_status	3 s
Revenue Over Time	stream_orders_live_gov (time-bucketed)	3 s
Latest Orders feed	stream_orders_raw ORDER BY ingest_time DESC LIMIT 25	3 s

4.3 Implementation choices

SQLAlchemy engine, not raw psycopg2. Cached via @st.cache_resource so the engine is reused across refreshes — avoids opening a new TCP connection every 3 seconds.

Plotly Express for charts. Concise API; built-in interactivity (hover, zoom) without extra code.

st.rerun() + time.sleep loop for refresh. Simpler than threading; Streamlit reruns the entire script on each tick, so all charts see consistent data from the same DB snapshot.

Environment variables for config. PG_HOST, PG_PORT, REFRESH_SECONDS, etc. read at startup; same code works in local Docker, on Windows host, or on Azure App Service.

5. Cloud Integration: Power BI & Azure

Three Azure / Microsoft cloud services extend the local stack. Each one is opt-in and is selected for the highest impact-per-effort ratio given a student plan ($100 credit + free Power BI Pro with .edu email).

Service	What it adds	Effort	Cost on student plan
Power BI streaming	Real-time enterprise BI dashboard	30 min	FREE with Power BI Pro
Azure App Service	Permanent public URL for Streamlit	1 hour	~$13/mo (credit-covered)
Azure Event Hubs	Managed Kafka, drop-in replacement	2 hours	~$11/mo Basic tier

5.1 Power BI streaming dataset (recommended first)

A Python **bridge script** (*powerbi_bridge.py*) polls Postgres every 5 seconds, builds a small payload per governorate × order_status combination, and posts it to a Power BI **streaming dataset** via the REST Push URL. Power BI then renders real-time tiles in the cloud workspace.

Dataset schema (created in Power BI UI):

Field	Type
timestamp	DateTime
governorate	Text
order_status	Text
orders_count	Number
revenue_egp	Number
avg_order_value	Number

5.2 Azure App Service for Streamlit

The Streamlit Dockerfile (Section A.4 in Appendix A) is published to Azure Container Registry, then deployed to Azure App Service (B1 tier). This gives the dashboard a permanent URL — for example, *https://egypt-ecomm.azurewebsites.net* — that does not depend on the laptop being on. PostgreSQL is exposed via a Cloudflare Tunnel (or replaced with Azure Database for PostgreSQL for a fully managed setup).

5.3 Azure Event Hubs (optional)

Azure Event Hubs exposes a Kafka-compatible surface. Migration requires changing only the *bootstrap_servers* string in *producer.py* and *consumer.py* to *<namespace>.servicebus.windows.net:9093* plus SASL_SSL credentials. The rest of the code is untouched — a key benefit of the Kafka API standard.

6. Public Deployment

For the graduation defense, the dashboard must be reachable from outside the developer's laptop. Three paths were evaluated:

Option	Public URL type	Setup time	Cost	When to use
A. ngrok tunnel	Random subdomain	5 min	Free	Demo day, one-time presentation
B. Streamlit Cloud + GitHub	Random subdomain	1 hour	Free	If you want a stable URL but stay free
C. Azure App Service	Permanent custom domain	1–2 hours	$13/mo	Permanent submission link for the report

6.1 Recommended path

Use Option A on demo day (one-line PowerShell command, public URL within seconds) and **Option C as the permanent URL in the written report**. Setting up Azure App Service in parallel with the defense ensures the link in the printed report remains live for grading.

6.2 ngrok one-liner

```
# After installing ngrok and adding your auth token:
ngrok http 8501

# Output:
# Session Status    online
# Forwarding        https://abc-12-34.ngrok-free.app -> http://localhost:8501
# Share that HTTPS URL with the panel.
```

7. Results & Metrics

The numbers below were measured on the running pipeline during a sustained 1-hour test on a Windows 16-CPU / 8 GB-RAM Docker Desktop host.

9 / 9 tables migrated	**~360 K** rows processed	**3,373** invalid rows removed	**5 s** micro-batch trigger
3 s dashboard refresh	**< 8 s** end-to-end latency	**3** live PG tables	**13** Docker containers

Beyond the numbers

The same Postgres backbone served two consumer surfaces (Streamlit + Power BI) at once with no observable performance impact on either — proving the dual-consumer pattern that production teams use to bridge engineering and business audiences.

8. Challenges & Solutions

● JMX Prometheus agent crash in Kafka CLI

Problem. The Confluent cp-kafka image binds port 1234 to a JMX Prometheus exporter for every Java process spawned inside the container. When kafka-topics is invoked via docker exec, the agent attempts to bind that port again and fails with java.net.BindException because the running broker is already using it.

✓ **Solution.** Created the orders_stream topic through the Kafka UI web interface at port 8090 instead, which avoids spawning a second Java process. For CLI operations, prepended unset KAFKA_OPTS to clear the agent configuration before running the command.

● Spark Structured Streaming has no native JDBC sink

Problem. writeStream.format("jdbc") simply does not exist in the Spark 3.5 API. Naive use raises AnalysisException at query start.

✓ **Solution.** Used the foreachBatch sink to convert each micro-batch into a regular DataFrame, then called the standard batch JDBC writer three times (raw, governorate aggregate, status aggregate). This is the officially recommended pattern in the Spark documentation.

● Pandas 2.2.2 compile failure on Python 3.13

Problem. Python 3.13 was released after pandas 2.2.2; no pre-built wheel exists. pip attempted to compile from C source via Meson and failed because Visual Studio build tools were not installed on the Windows machine.

✓ **Solution.** Removed strict version pinning in requirements.txt and instead let pip install the latest pandas, which has Python 3.13 wheels. Captured the resolved versions with pip freeze for reproducibility.

● PowerShell context resets between windows

Problem. Every fresh PowerShell window opens in C:\WINDOWS\System32 with no virtual environment active. This caused repeated failures like 'streamlit not recognized' and 'Access denied' when writing files (because System32 is protected).

✓ **Solution.** Documented a 'cd + activate' ritual that must precede every command in every new terminal. Added a helper PowerShell command to rename each window's title bar so terminals don't get confused at demo time.

● pgAdmin login credentials unknown

Problem. pgAdmin has its own master account, separate from the PostgreSQL admin/admin credentials. The default values depend on environment variables set when the container was first started.

✓ **Solution.** Used Get-Content … | docker exec -i postgres_general psql to execute the SQL directly inside the Postgres container, bypassing the pgAdmin login entirely. This is also faster for repeated runs.

9. Future Work

→ **Schema Registry + Avro.** Migrate from JSON to Avro with Confluent Schema Registry to enforce producer/consumer contracts and reduce payload size.

→ **Multi-broker Kafka.** Replicate orders_stream across 3 brokers with replication-factor=3 for fault tolerance.

→ **Personalization model.** Train a recommendation model on the silver layer (collaborative filtering or content-based) and serve it via streaming Spark MLlib.

→ **Cloud-native rebuild.** Replace the local stack with Azure Event Hubs + Azure Stream Analytics + Power BI streaming, deployed via Bicep infrastructure-as-code.

→ **Airflow orchestration of streaming snapshots.** Add a daily Airflow DAG that snapshots streaming aggregates back into the batch silver layer for long-term trend analysis.

→ **Arabic NLP on reviews.** Apply sentiment analysis to review_comment_message (Arabic text) and surface sentiment as a dashboard tile.

→ **Geo visualization.** Use the geolocation table to plot orders on a real Egypt map (Folium / Mapbox in Streamlit, ArcGIS in Power BI).

10. Conclusion

This project demonstrates a complete, working big-data pipeline that spans batch ingestion, distributed cleaning, real-time streaming, dual-surface visualization, and optional cloud deployment — built and documented end-to-end. The two-dashboard pattern (Streamlit + Power BI) on a shared Postgres backbone mirrors how modern data teams structure analytics in production. Every component is containerized and reproducible, every design decision is documented, and every challenge encountered along the way is logged with its resolution.

The result is a graduation project that goes well beyond batch ETL: it shows readiness to build, operate, and reason about production streaming systems.

Appendix A — Code Listings

A.1 producer.py (key excerpt)

```
def main():
    pool = fetch_order_pool(POOL_SIZE)
    producer = KafkaProducer(
        bootstrap_servers=KAFKA_BOOTSTRAP,
        value_serializer=lambda v: json.dumps(v).encode("utf-8"),
        key_serializer=lambda k: k.encode("utf-8"),
        acks="all", retries=5, linger_ms=50,
    )
    while _running:
        row = random.choice(pool)
        event = to_event(row)
        producer.send(TOPIC, key=event["order_id"], value=event)
        time.sleep(random.uniform(0.2, 1.5))
    producer.flush(); producer.close()
```

A.2 consumer.py (Spark builder)

```
def build_spark():
    return (SparkSession.builder
        .appName("EgyptEcommStreaming")
        .config("spark.sql.shuffle.partitions", "4")
        .config("spark.sql.session.timeZone", "Africa/Cairo")
        .getOrCreate())

raw_kafka = (spark.readStream
    .format("kafka")
    .option("kafka.bootstrap.servers", KAFKA_BOOTSTRAP)
    .option("subscribe", TOPIC)
    .option("startingOffsets", "latest")
    .option("failOnDataLoss", "false")
    .load())

parsed = (raw_kafka
    .selectExpr("CAST(value AS STRING) AS json_str")
    .select(from_json(col("json_str"), EVENT_SCHEMA).alias("d"))
    .select("d.*")
    .withColumn("event_time", to_timestamp(col("event_time"))))

query = (parsed.writeStream
    .foreachBatch(foreach_batch)
    .outputMode("append")
    .option("checkpointLocation", CHECKPOINT_DIR)
    .trigger(processingTime="5 seconds")
    .start())
```

A.3 app.py (Streamlit, KPI cards)

```
@st.cache_resource
def get_engine():
    url = f"postgresql+psycopg2://{PG_USER}:{PG_PASS}@{PG_HOST}:{PG_PORT}/{PG_DB}"
    return create_engine(url, pool_pre_ping=True)

def q(sql):
    with get_engine().connect() as conn:
        return pd.read_sql(text(sql), conn)

kpis = q(f"""
    SELECT COUNT(*) AS total_orders,
           SUM(payment_value_egp) AS total_revenue,
           COUNT(DISTINCT customer_governorate) AS active_govs,
           AVG(payment_value_egp) AS avg_order_value
    FROM stream_orders_raw
    WHERE ingest_time > NOW() - INTERVAL '{LOOKBACK_MIN} minutes'
""")

c1, c2, c3, c4 = st.columns(4)
c1.metric("Orders",     f"{int(kpis['total_orders'][0]):,}")
c2.metric("Revenue",    f"{float(kpis['total_revenue'][0]):,.0f}")
c3.metric("Govs",       int(kpis['active_govs'][0]))
c4.metric("Avg value", f"{float(kpis['avg_order_value'][0]):,.2f}")

time.sleep(REFRESH_SECONDS); st.rerun()
```

A.4 Streamlit Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
RUN apt-get update && apt-get install -y --no-install-recommends \
      gcc libpq-dev curl && rm -rf /var/lib/apt/lists/*
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 8501
HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
  CMD curl -fsS http://localhost:8501/_stcore/health || exit 1
CMD ["streamlit", "run", "app.py", \
     "--server.port=8501", "--server.address=0.0.0.0", \
     "--server.headless=true"]
```

A.5 powerbi_bridge.py (REST push)

```
def push_to_powerbi(rows):
    if not rows: return True
    resp = requests.post(POWERBI_PUSH_URL, json=rows, timeout=10)
    return resp.status_code in (200, 202)

def main():
    engine = get_engine()
    while True:
        df = fetch_recent_rollup(engine)
        rows = [{
            "timestamp": datetime.utcnow().isoformat(),
            "governorate": r["governorate"],
            "order_status": r["order_status"],
            "orders_count": int(r["orders_count"]),
            "revenue_egp": float(r["revenue_egp"]),
            "avg_order_value": float(r["avg_order_value"]),
```

```
        } for _, r in df.iterrows()]
        push_to_powerbi(rows)
        time.sleep(POLL_INTERVAL_SEC)
```

Appendix B — SQL Schemas

B.1 Streaming tables (init_streaming_tables.sql)

```
CREATE TABLE stream_orders_raw (
    order_id              VARCHAR(64),
    customer_id           VARCHAR(64),
    customer_governorate  VARCHAR(64),
    customer_state        VARCHAR(64),
    customer_city         VARCHAR(64),
    order_status          VARCHAR(32),
    product_id            VARCHAR(64),
    price_egp             DOUBLE PRECISION,
    freight_value_egp     DOUBLE PRECISION,
    payment_type          VARCHAR(32),
    payment_installments  INTEGER,
    payment_value_egp     DOUBLE PRECISION,
    event_time            TIMESTAMP,
    batch_id              BIGINT,
    ingest_time           TIMESTAMP DEFAULT NOW()
);
CREATE INDEX idx_raw_event_time ON stream_orders_raw(event_time DESC);
CREATE INDEX idx_raw_batch      ON stream_orders_raw(batch_id);
CREATE INDEX idx_raw_gov        ON stream_orders_raw(customer_governorate);

CREATE TABLE stream_orders_live_gov (
    customer_governorate VARCHAR(64),
    orders_count         BIGINT,
    total_revenue_egp    DOUBLE PRECISION,
    batch_id             BIGINT,
    batch_time           TIMESTAMP
);

CREATE TABLE stream_orders_live_status (
    order_status         VARCHAR(32),
    orders_count         BIGINT,
    total_revenue_egp    DOUBLE PRECISION,
    batch_id             BIGINT,
    batch_time           TIMESTAMP
);

CREATE VIEW v_stream_kpis AS
SELECT COUNT(*)                              AS total_orders_last_5m,
       SUM(payment_value_egp)                AS total_revenue_last_5m,
       COUNT(DISTINCT customer_governorate)  AS active_govs_last_5m
FROM stream_orders_raw
WHERE ingest_time > NOW() - INTERVAL '5 minutes';
```

Appendix C — Setup & Operations Guide

C.1 Prerequisites

- Docker Desktop on Windows 10/11 (8 GB RAM minimum, 16 GB recommended)
- Python 3.10, 3.11, or 3.12 with pip and venv (for the Windows-host Streamlit and producer)
- Shared volume mounted at F:\Docker\shared → /data inside every container
- All 13 containers from the project docker-compose.yaml running and healthy
- Free disk space ≥ 20 GB on the F: drive

C.2 Run order (3 terminals)

Terminal 1 — Spark consumer (inside jupyter container)

```
docker exec -it jupyter bash
bash /data/streaming/spark_consumer/submit.sh
# Wait for: [Streaming] Query started
```

Terminal 2 — Producer (Windows host, venv active)

```
cd F:\streamlit_local
.\venv\Scripts\Activate.ps1
$env:USE_LOCAL = "1"
python producer.py
```

Terminal 3 — Streamlit (Windows host, venv active)

```
cd F:\streamlit_local
.\venv\Scripts\Activate.ps1
$env:PG_HOST = "localhost"
streamlit run app.py
```

Optional Terminal 4 — Power BI bridge

```
cd F:\streamlit_local
.\venv\Scripts\Activate.ps1
$env:POWERBI_PUSH_URL = ""
python powerbi_bridge.py
```

C.3 Health checks

Kafka topic exists	`docker exec broker bash -c "unset KAFKA_OPTS && kafka-topics --bootstr`
PostgreSQL tables present	`docker exec postgres_general psql -U admin -d sessiondb - "\dt stream`
Spark UI	`http://localhost:4040 (active query, micro-batch progress)`
Kafka UI	`http://localhost:8090 (messages flowing on orders_stream)`
Streamlit dashboard	`http://localhost:8501 (KPIs increment every 3 seconds)`
Airflow UI	`http://localhost:8089 (login: airflow / airflow)`

C.4 Shutdown order

Reverse of startup. Press **Ctrl+C** in each terminal — first Streamlit and the producer, then the Spark consumer (so it flushes its checkpoint). Container stack can stay running; **docker-compose down** is only needed if you want to free the resources entirely.

Project Summary

Component	Technology	Status
Source Data	9 CSV files, Egyptian Olist adaptation	✓Complete
Bronze Layer	Spark → HDFS Parquet	✓Complete
Silver Layer	Spark transformations → HDFS Parquet	✓Complete
Gold Layer — Delivery	Spark → PostgreSQL (delivery_performance schema)	✓Complete
Gold Layer — Churn	Spark → PostgreSQL (customer_churn schema)	✓Complete
Orchestration	Apache Airflow DAG (4 tasks, sequential)	✓Complete
Dashboard — Delivery	Power BI, 3 pages, 6 visuals each	✓Complete
Dashboard — Churn	Power BI, 3 pages, 5 visuals each	✓Complete
Dashboard — Streamlit	Kafka ·Streamlit	✓Complete
Infrastructure	12 Docker containers, WSL2	✓Complete